

complessità più verso $O(n \log n)$; ma sicuramente ci sono dei casi "difficili" che rendono la complessità comunque $O(n^2)$.

Esercizio B1 – Commissioni di laurea – Punti ≥ 8

Il problema può essere risolto tramite una rete di flusso.

Creiamo i seguenti nodi:

- una sorgente s ;
- un nodo per ogni professore, $p_1 \dots p_n$;
- due nodi per ogni commissione i , $1 \leq i \leq k$, chiamati c_i^2 e c_i^3 ; c_i^2 serve a raccogliere i due professori esperti, c_i^3 serve a raccogliere gli altri 3 professori;
- un pozzo t

Creiamo i seguenti archi:

- colleghiamo s a ogni professore, con peso 1 (in quanto può partecipare a una sola commissione) (n archi);
- colleghiamo ogni professore ai nodi c_i^2 , dove i è l'area didattica in cui il professore è esperto (n archi), con peso 1
- colleghiamo ogni professore ai nodi c_i^3 (nk archi), con peso 1;
- colleghiamo tutti i nodi c_i^2 e c_i^3 al pozzo ($2k$ archi), con peso 2 e 3, rispettivamente.

La dimensione di V ed E è la seguente:

$$|V| = 2 + n + 2k$$

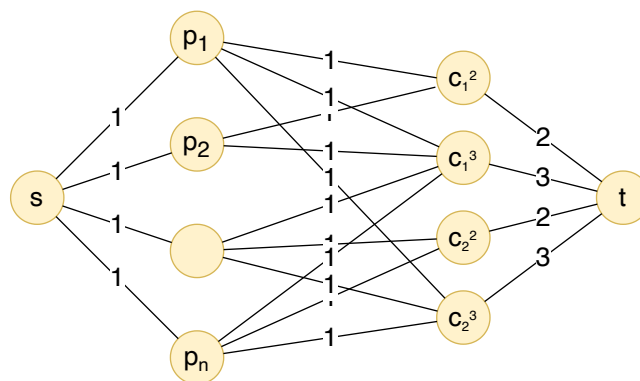
$$|E| = n + n + nk + 2k = 2n + nk + 2k$$

Il flusso massimo è limitato superiormente sia da n che da $5k$; condizione perché si possa formare le commissioni seguendo queste regole è che $n \geq 5k$, quindi useremo il limite più stretto che è $5k$.

Utilizzando il limite di Ford-Fulkerson e tenuto conto che il flusso massimo è limitato superiormente sia da n , il costo totale dell'algoritmo proposto è

$$O(5k \cdot (|V| + |E|)) = O(5k(2 + 3n + nk + 4k)) = O(nk^2)$$

La figura seguente mostra un piccolo esempio con n professori e due commissioni.



Esercizio B2 – Tutti i cicli! – Punti ≥ 10

La soluzione può essere ottenuta adattando una visita in profondità e utilizzando la tecnica del backtracking. Si noti innanzitutto che ogni ciclo che passa attraverso *start* può essere espresso come un ciclo che parte da *start* e arriva a *start*. Quindi, eseguiamo una visita in profondità, segnando come visitato ogni nodo che si raggiunge e marcandolo come non visitato una volta completata la sua visita.

Viene utilizzato uno stack aggiuntivo, in cui vengono memorizzati i nodi visitati finora. Quando si incontra di nuovo il nodo di partenza, si stampa il contenuto dello stack (assumiamo che venga stampato dal fondo alla cima) e poi si stampa di nuovo il nodo di partenza, per chiudere il ciclo.

boolean printCycles(GRAPH G , NODE $start$)
STACK $stack = \text{Stack}()$ boolean [] $visited = \text{new boolean}[1 \dots G.n] = \{\text{false}\}$ printCyclesRec($G, start, start, stack, visited$)
printCyclesRec(GRAPH G , NODE $start$, NODE $curr$, STACK $stack$, boolean [] $visited$)
$stack.\text{push}(curr)$ $visited[curr] = \text{true}$ foreach $next \in G.\text{adj}(curr)$ do if $next == start$ then print $stack, start$ else if not $visited[next]$ then printCycleRec ($G, start, next, stack$) $visited[curr] = \text{false}$ $stack.\text{pop}()$

Analisi complessità, semplice

- Come caso ottimo, consideriamo un albero o addirittura una foresta. Il costo dell'algoritmo sarà lineare nel numero dei nodi (il numero degli archi è inferiore o uguale a $n - 1$), perché verrà effettuata una semplice visita e non verrà stampato nulla. Possiamo dire quindi che l'algoritmo ha costo $\Omega(n)$.
- Come caso pessimo, consideriamo un grafo completo, dove tutti i nodi sono connessi a tutti gli altri nodi. Partendo da un nodo, esistono $(n - 1)!$ possibili cicli di $n - 1$ nodi, che partono da $start$, attraversano una permutazione di $n - 1$ nodi e ritornando a $start$. Poiché la stampa dei cicli costa $O(n)$, nel caso pessimo il costo è $T(n) = \Omega(n!)$; il limite è inferiore perché non stiamo considerando cicli di lunghezza inferiore, che dovrebbero essere comunque contati. Tuttavia, come analisi è sufficiente, perché ci dice che l'algoritmo ha costo almeno fattoriale nel caso pessimo.

Analisi complessità, completa (non necessaria) È possibile osservare che il numero di cicli contenenti un nodo in un grafo completo può essere calcolato nel modo seguente:

- $(n - 1)$ cicli di lunghezza 2 ($start \rightarrow u \rightarrow start$, con $u, start$ distinti);
- $(n - 1) \cdot (n - 2)$ cicli di lunghezza 3 ($start \rightarrow u \rightarrow v \rightarrow start$, con $u, v, start$ distinti);
- ...
- $(n - 1)!$ cicli di lunghezza $n - 1$ (partendo da $start$, attraversando una permutazione di $n - 1$ nodi e ritornando a $start$)

Il numero di cicli effettivamente stampati nel caso di un grafo completo è quindi pari a:

$$C(n) = \sum_{k=1}^{n-1} \frac{(n-1)!}{(n-1-k)!}$$

Il numero di cicli da stampare cresce, partendo da $n = 2$, in questo modo: 1, 4, 15, 64, 325, 1956, 13699, 109600, 986409. Questa sequenza corrisponde alla sequenza <https://oeis.org/A007526> registrata nel sito OIS; in effetti, corrisponde al numero di permutazioni di sottoinsiemi non vuoti di $n - 1$ elementi. Utilizzando la formula di Joseph K. Horn citata nella pagina, si ottiene una complessità pari a:

$$C(n) = \lfloor e \cdot (n - 1)! - 1 \rfloor$$

Possiamo quindi concludere che il costo computazionale nel caso pessimo è $O(n!)$, derivante da $O(n)$ per la stampa dei cicli e $O((n - 1)!)$ per il numero di cicli. Combinato con il risultato che $T(n) = \Omega(n!)$ nel caso pessimo, si può concludere che $T(n) = \Theta(n!)$ sempre nel caso pessimo.

Esercizio B3 – Quanti quadrati! – Punti ≥ 12

Questo problema è simile a un problema discusso nelle esercitazioni in aula: determinare la dimensione della più grande sottomatrice quadrata presente nella matrice.

Nel peggiore dei casi, può essere risolto in tempo $O(n^5)$: per ognuna delle n^2 celle (i, j) e per ogni possibile dimensione k delle n dimensioni, si verifica se esiste un quadrato di dimensione $k \times k$ la cui cella in alto a sinistra è in posizione (i, j) , in tempo $O(n^2)$. In altre parole, mediante cinque cicli annidati.

È possibile trovare una soluzione in $O(n^4)$ nel modo seguente: per ognuna delle n^2 celle (i, j) , si cerca la dimensione k del più grande quadrato la cui cella in alto a sinistra è in posizione (i, j) e si tiene conto del fatto che, se esiste un quadrato di dimensione k , allora esistono anche quadrati di dimensione $k-1, k-2, \dots, 1$ che iniziano in (i, j) . Questo implica l'utilizzo di quattro cicli annidati.

È possibile trovare una soluzione in $O(n^3)$ definendo una relazione di programmazione dinamica $DP[i][j][k]$, che stabilisce se esiste una sottomatrice quadrata di dimensione k il cui angolo in basso a destra si trova in posizione (i, j) .

Riutilizziamo il codice discusso nell'esercitazione, che calcolava una matrice $DP[1 \dots n][1 \dots n]$ memorizzando in $DP[i][j]$ la dimensione della più grande sottomatrice quadrata il cui angolo in basso a destra si trova nella cella (i, j) .

Tutte le celle (i, j) con valore $A[i][j] = 0$ hanno una dimensione massima pari a 0. Per le celle con valore 1, se si trovano sui bordi superiore ($i = 1$) o sinistro ($j = 1$), il loro valore è pari a 1. Altrimenti, per formare un quadrato di dimensione k che termina in (i, j) , è necessario che esistano quadrati di dimensione almeno $k-1$ nelle celle "sopra" $(i-1, j)$, "sopra a sinistra" $(i-1, j-1)$ e "a sinistra" $(i, j-1)$; si prende quindi il valore minimo tra queste celle e si somma 1.

$$DP[i][j] = \begin{cases} 0 & \text{se } A[i][j] = 0 \\ 1 & \text{se } A[i][j] = 1 \wedge (i = 1 \vee j = 1) \\ \min\{DP[i-1][j], \\ DP[i-1][j-1], \\ DP[i][j-1]\} + 1 & \text{altrimenti} \end{cases}$$

È possibile calcolare DP con due cicli annidati. Una volta stabilita la dimensione massima delle sottomatrici che terminano in ognuna delle celle (i, j) , si può utilizzare l'osservazione proposta per passare dalla soluzione $O(n^5)$ alla soluzione $O(n^4)$, ovvero che se esiste una sottomatrice quadrata di dimensione $DP[i][j]$ che termina in posizione (i, j) , allora esistono anche sottomatrici quadrate che terminano in posizione (i, j) di dimensione $DP[i][j]-1, DP[i][j]-2, \dots, 1$. Dobbiamo quindi contare $DP[i][j]$ quadrati per ognuna delle celle (i, j) . È quindi sufficiente restituire la somma della matrice DP .

```
int maxSquare(boolean[][] A, int n)
```

```
    int[][] DP = new int[1...n][1...n]
```

```
    for i = 1 to n do
```

```
        DP[i][1] = A[i][1]
        DP[1][i] = A[1][i]
```

```
    for i = 2 to n do
```

```
        for j = 2 to n do
```

```
            if A[i][j] == 0 then
```

```
                DP[i][j] = 0
```

```
            else
```

```
                DP[i][j] = min(DP[i-1][j], DP[i-1][j-1], DP[i][j-1]) + 1
```

```
    return sum(A, n)
```

```
% Return matrix sum,  $\Theta(n^2)$ 
```

La complessità di questo algoritmo è ovviamente $\Theta(n^2)$.